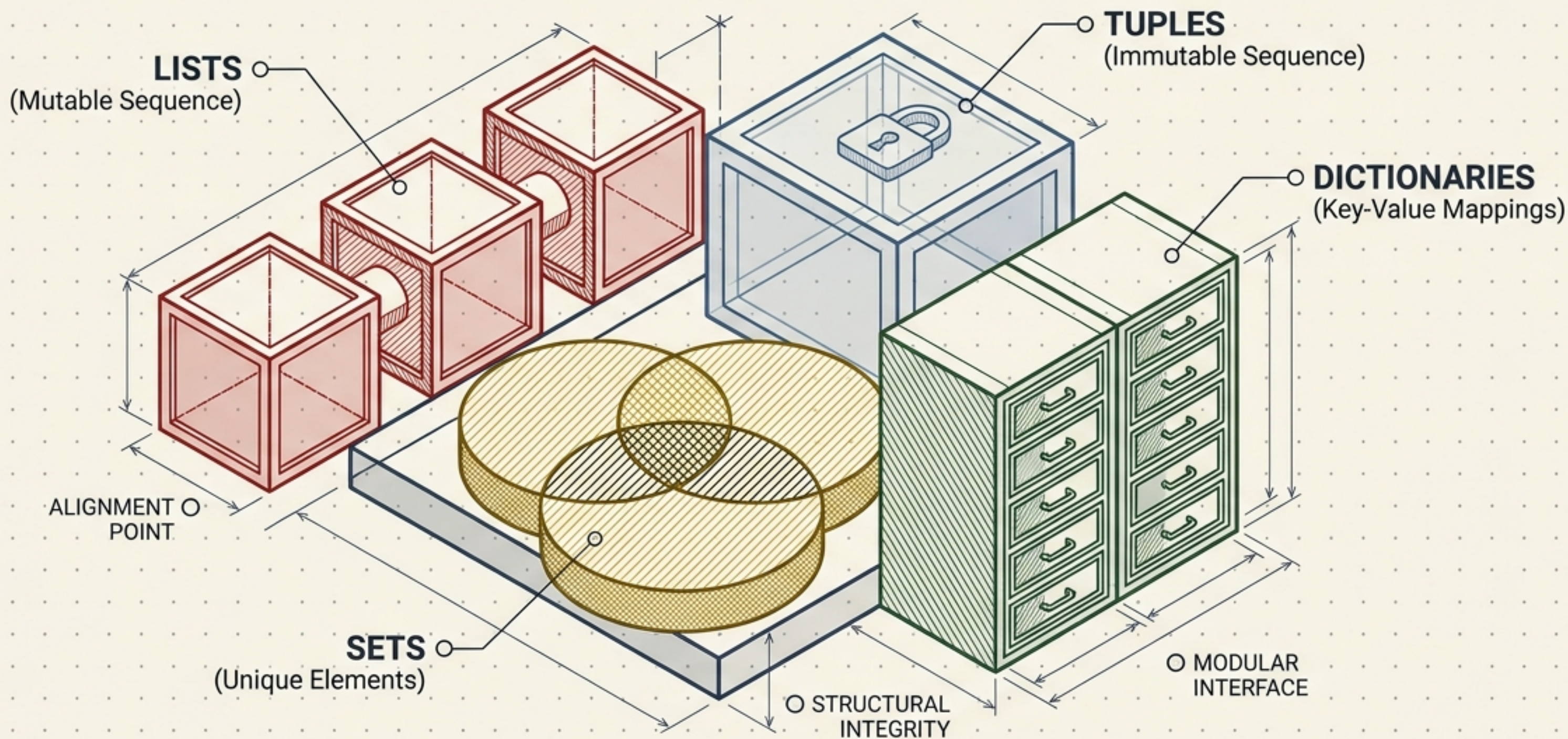


The Python Collections Architecture

Master Built-In Containers and Custom Extensions

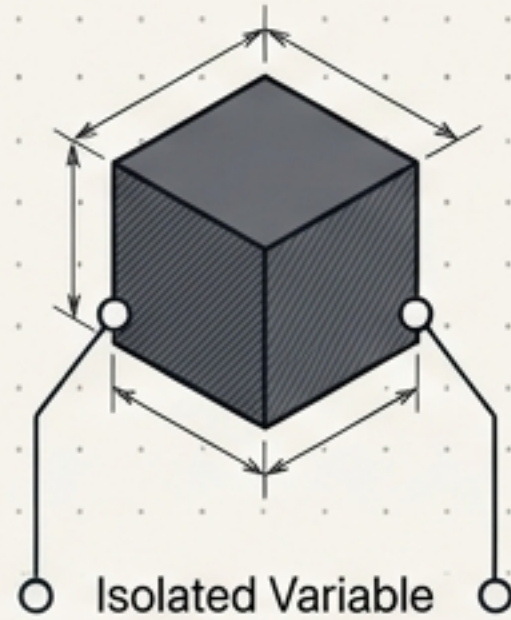


The Foundation of Data Management

Container data types form the bedrock of Python programs. They replace isolated variables with scalable architecture.

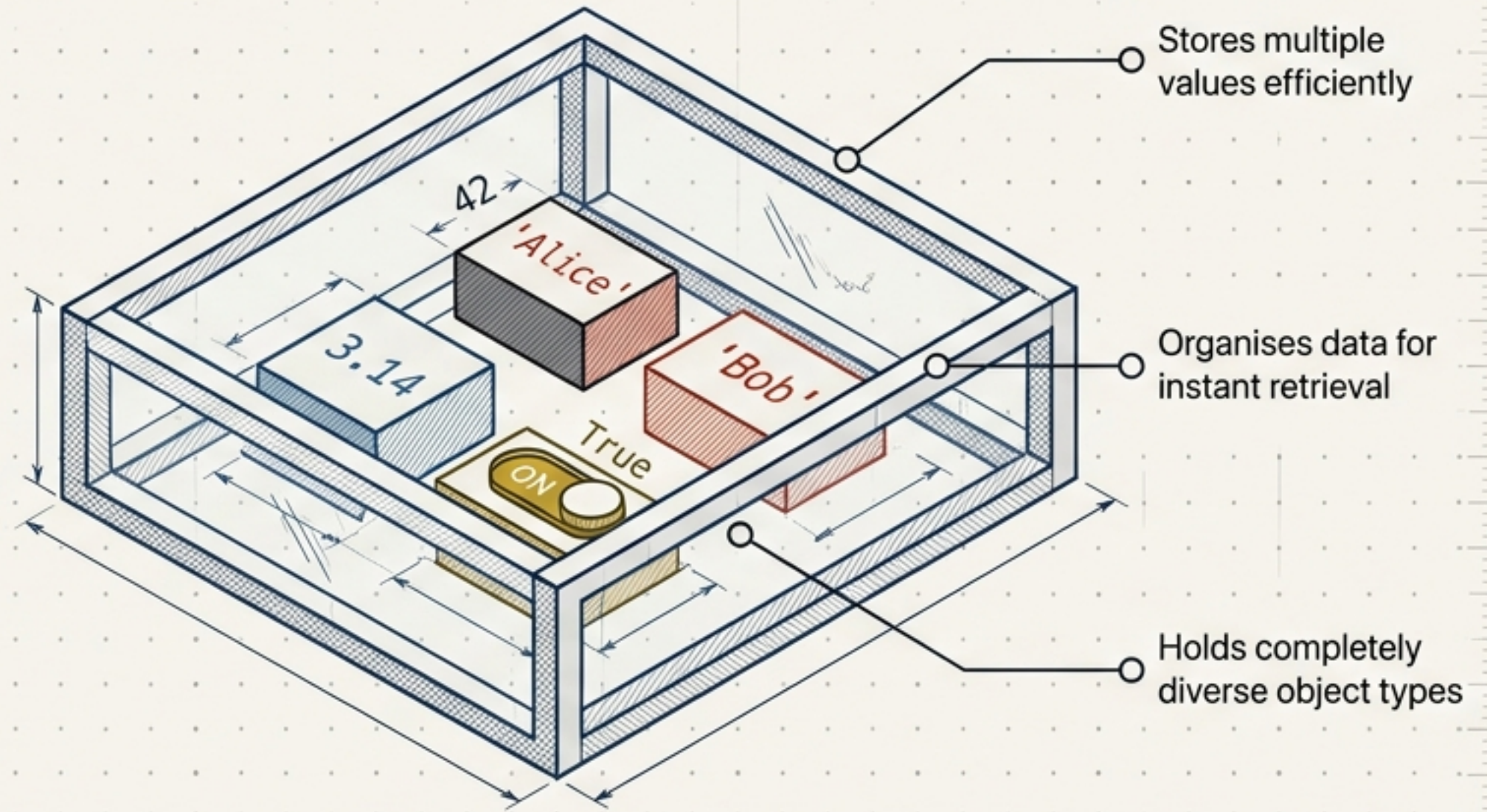
The Limitation

```
x = 5
```



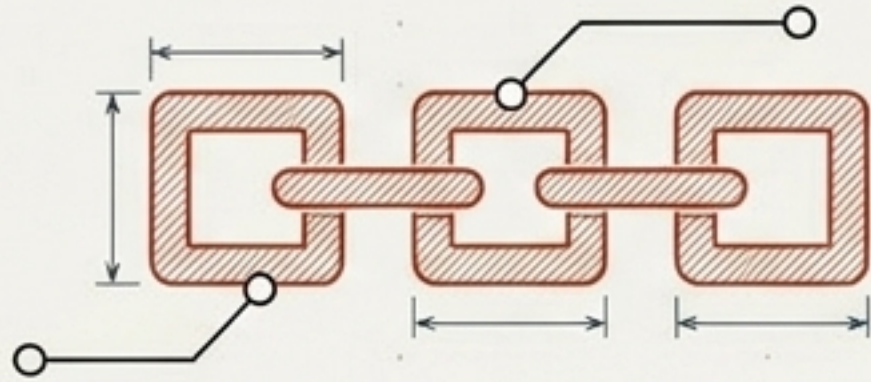
The Solution

```
data = [42, 'Alice', 3.14, True, 'Bob']
```



The Built-In Toolkit: The Big Four

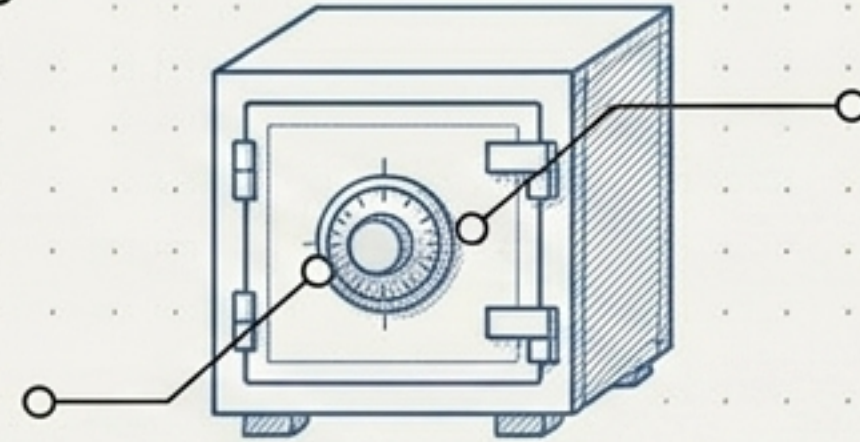
Lists



Syntax: `[]`

```
numbers = [1, 2, 3]
```

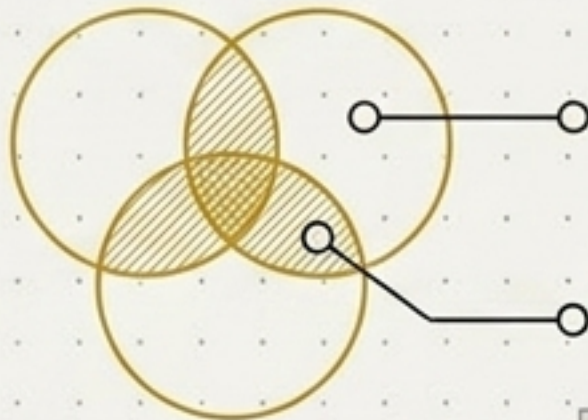
Tuples



Syntax: `()`

```
person = ('John', 25)
```

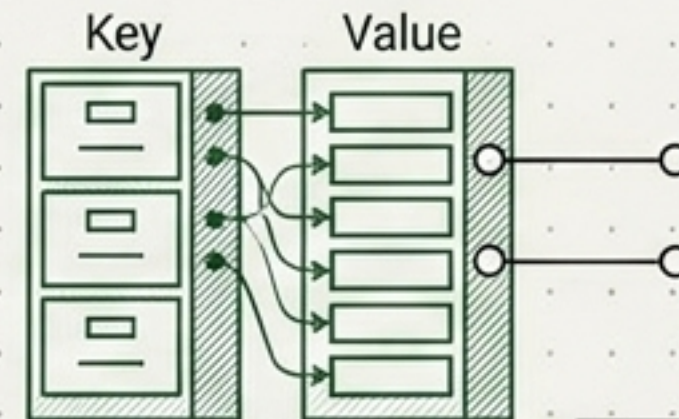
Sets



Syntax: `{}`

```
colors = {'red', 'blue'}
```

Dictionaries

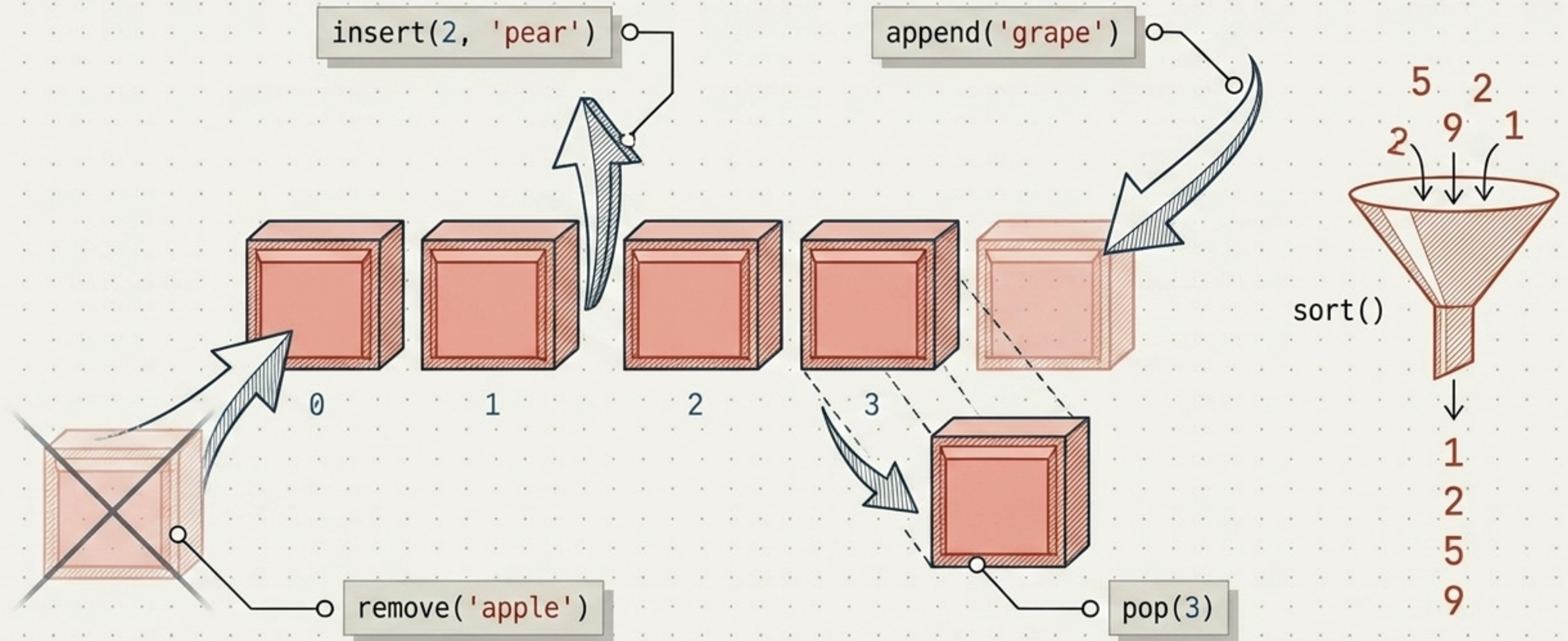


Syntax: `{key: value}`

```
student = {'name': 'Alice'}
```

Lists | The Dynamic Sequence

Ordered, mutable collections accessed by index. Designed for dynamic, sequential data.



List Comprehensions | The Functional Transformer

Reduces boilerplate code and accelerates data processing.

```
squares = []  
for x in range(5):  
    squares.append(x * x)
```

Before:
Standard Loop

Refactored

```
squares = [x * x for x in range(5)]
```

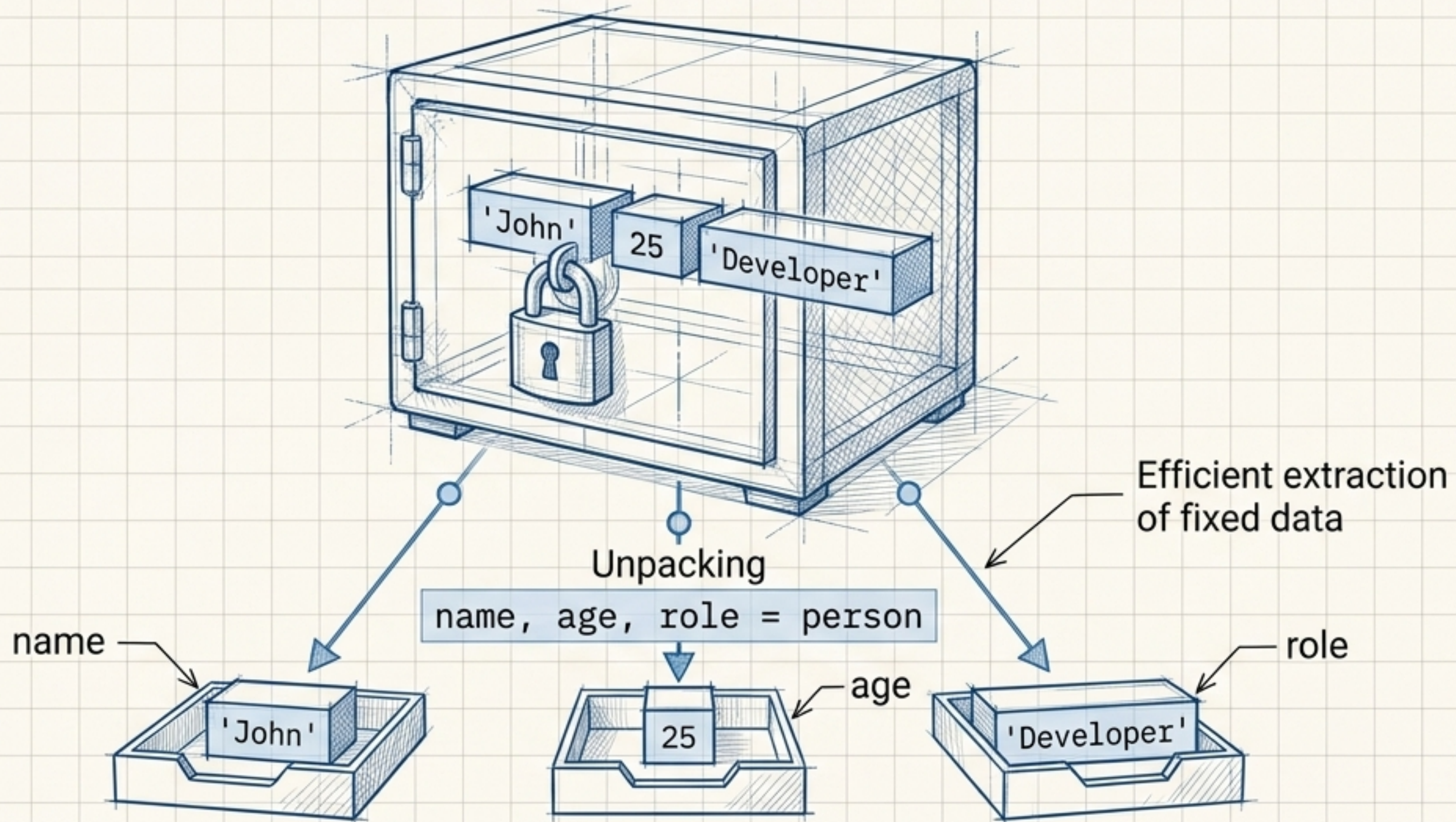
Transformation
Logic

Iteration
Engine

Tuples | The Immutable Vault

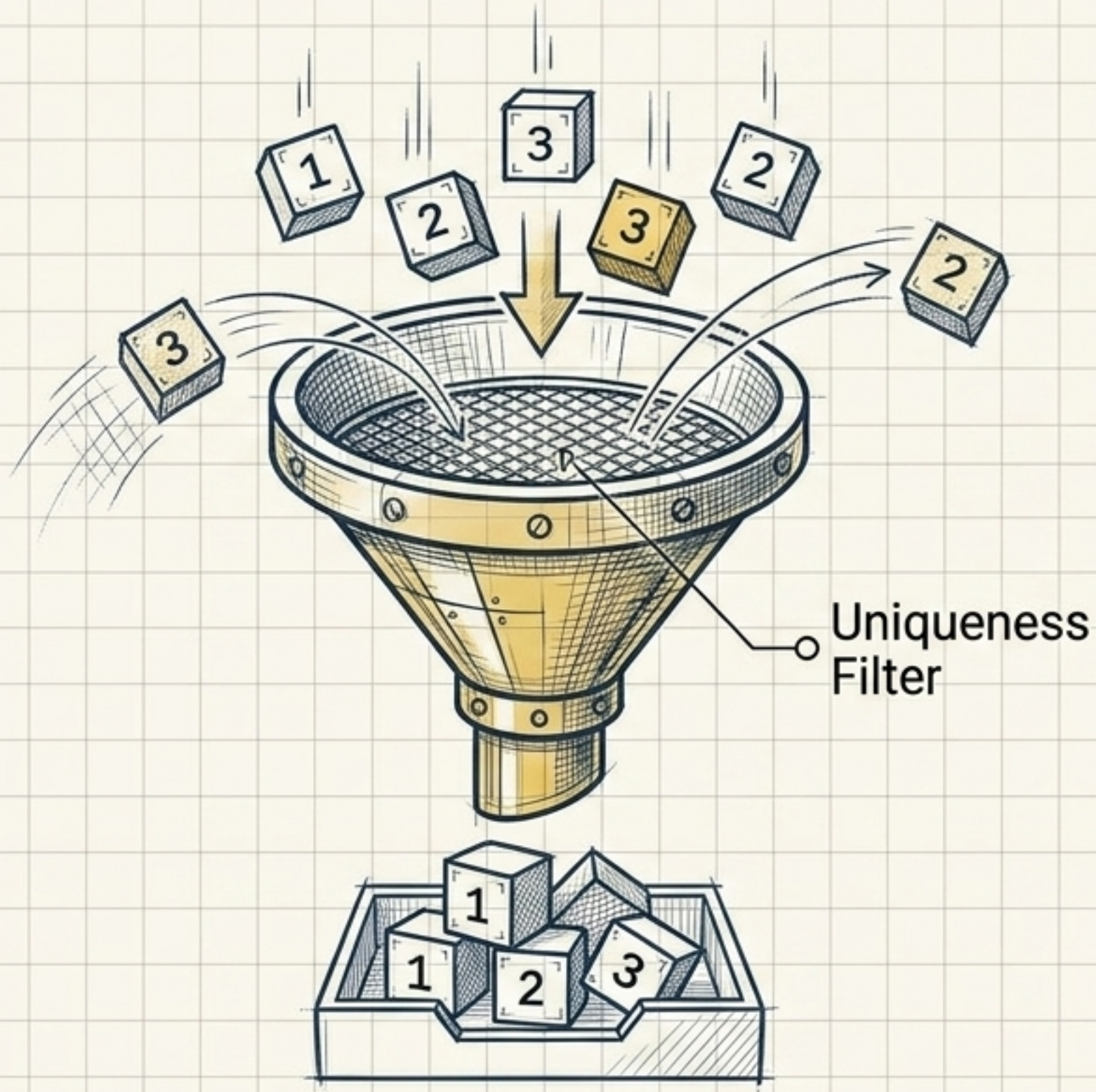
Ordered, fixed collections. Highly memory-efficient and tamper-proof after creation.

```
person = ('John', 25, 'Developer')
```



Sets | The Uniqueness Engine

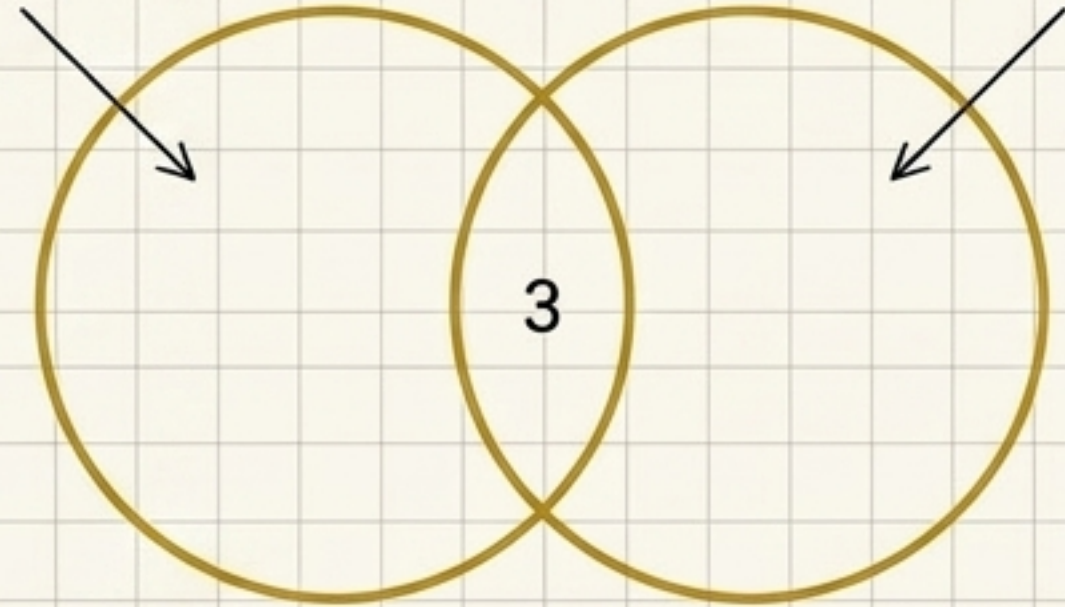
Unordered collections that automatically enforce uniqueness and power mathematical operations.



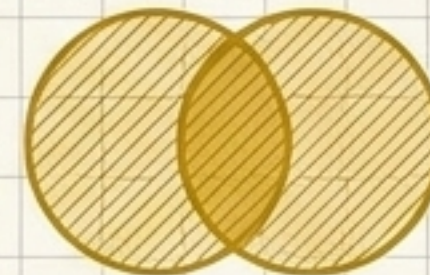
The Math

set1 = {1, 2, 3}

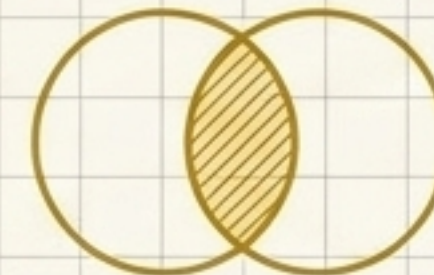
set2 = {3, 4, 5}



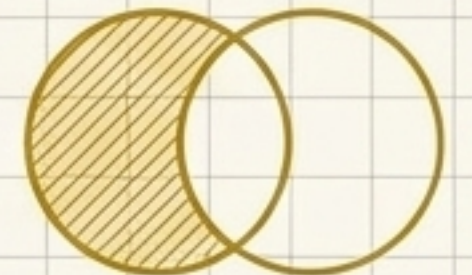
union()



intersection()

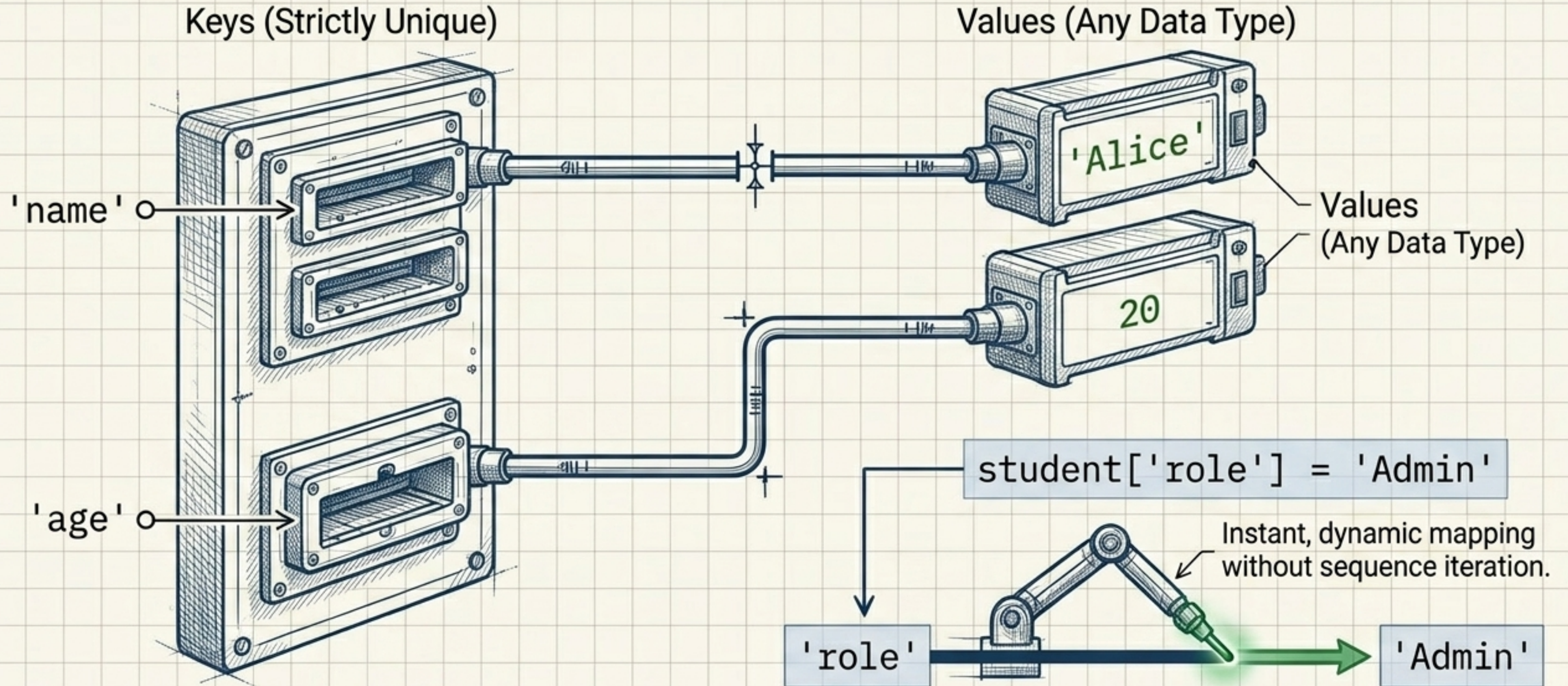


difference()



Dictionaries | The Key-Value Switchboard

Optimized mapping structures for instant data retrieval via unique keys.

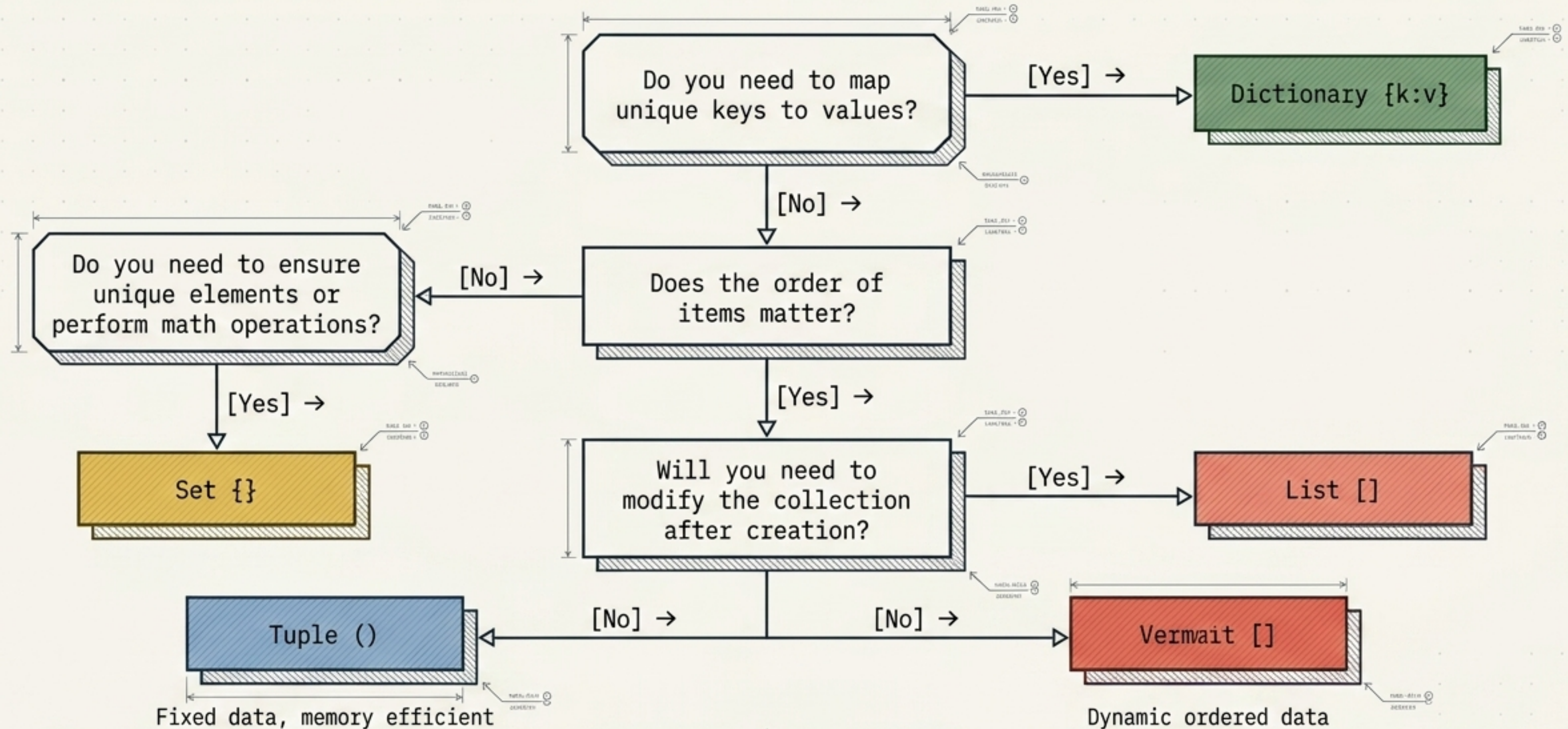


The Container Diagnostic Matrix

Container	Ordered?	Mutable?	Duplicates?	Fast Lookup?
List []	✓ Yes	✓ Yes	✓ Yes	✗ No
Tuple ()	✓ Yes	✗ No	✓ Yes	✗ No
Set {}	✗ No	✓ Yes	✗ No	✓ Yes
Dict {k:v}	✓ Insert Order	✓ Yes	✗ Keys / ✓ Values	✓ Yes

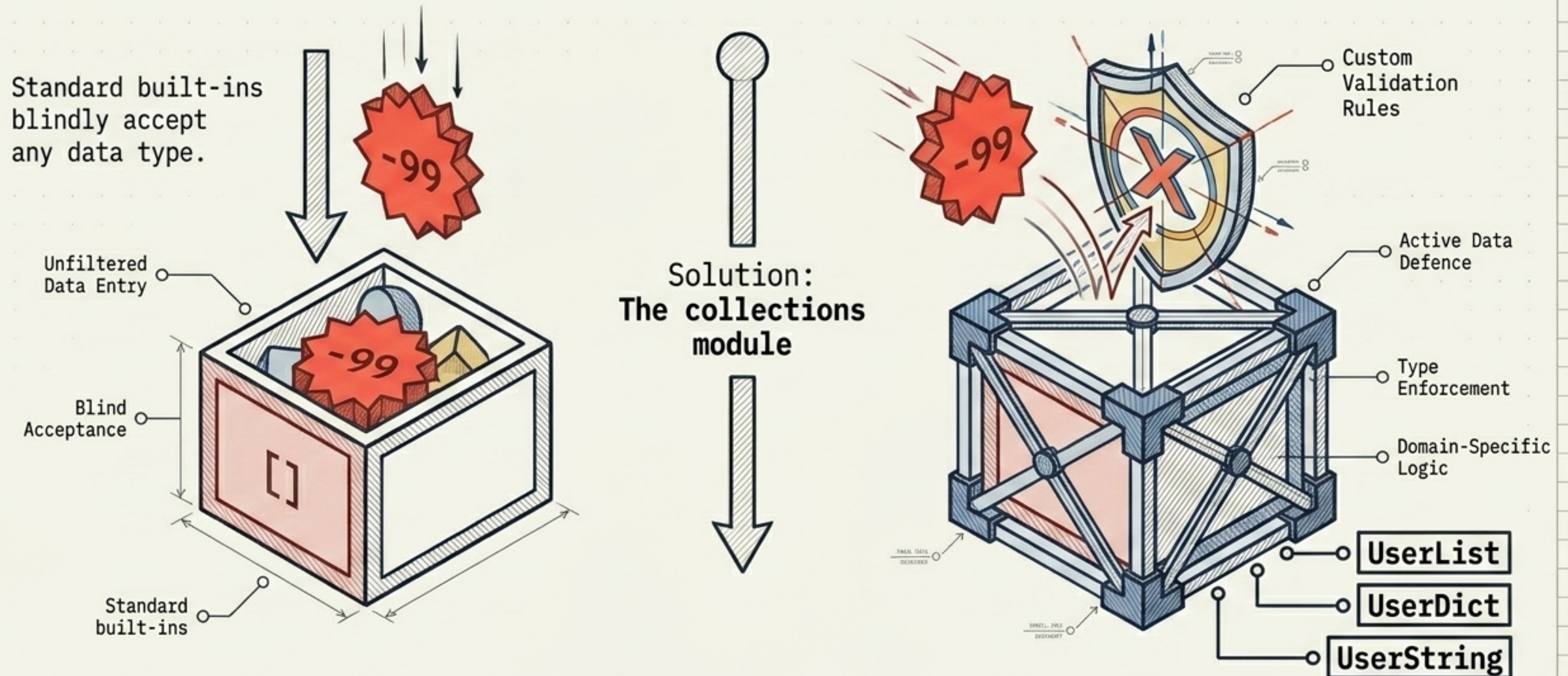
The Architecture Selection Algorithm

Optimize performance and readability by selecting the structurally correct tool.



Beyond the Built-ins: The Custom Workshop

What if you need validation, domain-specific behaviours, or strict typing?

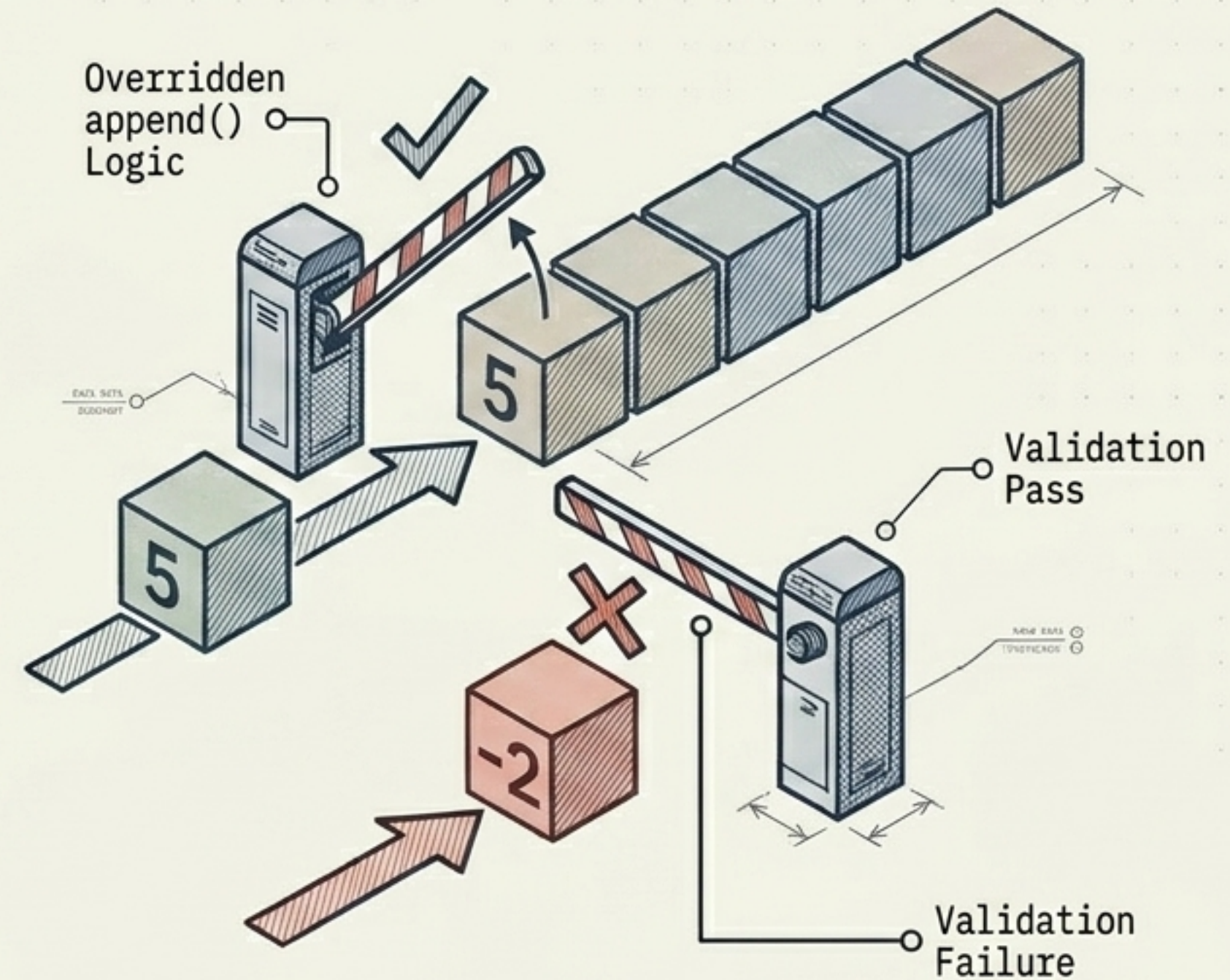


UserList | Engineering a Guarded Sequence

Extending the list sequence securely to enforce strict validation rules upon entry.

```
from collections import UserList

class PositiveNumbers(UserList):
    def append(self, item):
        if item > 0:
            super().append(item)
```

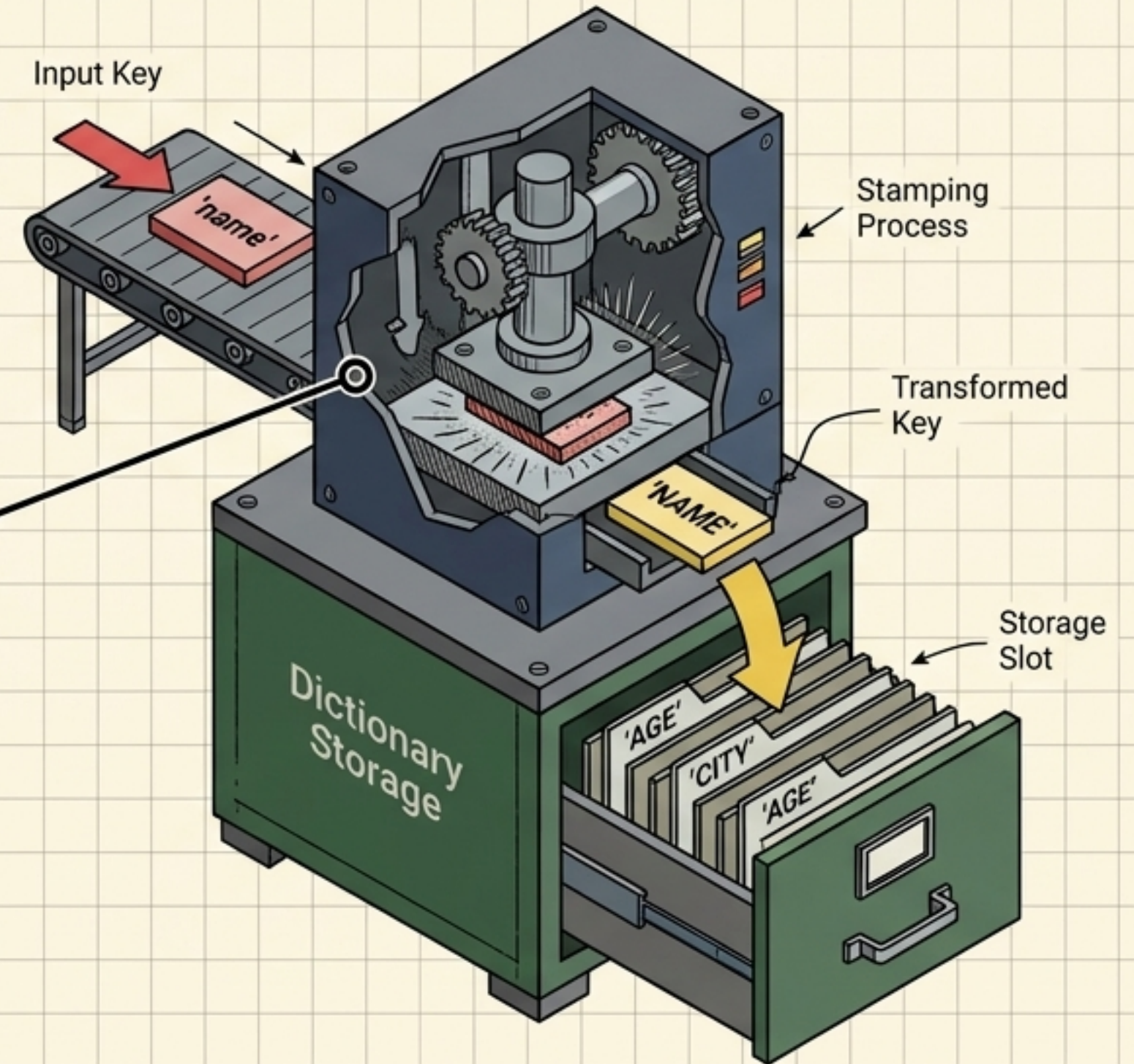


UserDict | Customising the Map

Building automated processing mechanisms directly into standard mapping structures.

```
from collections import UserDict

class UpperDict(UserDict):
    def __setitem__(self, key, value):
        super().__setitem__(key.upper(), value)
```



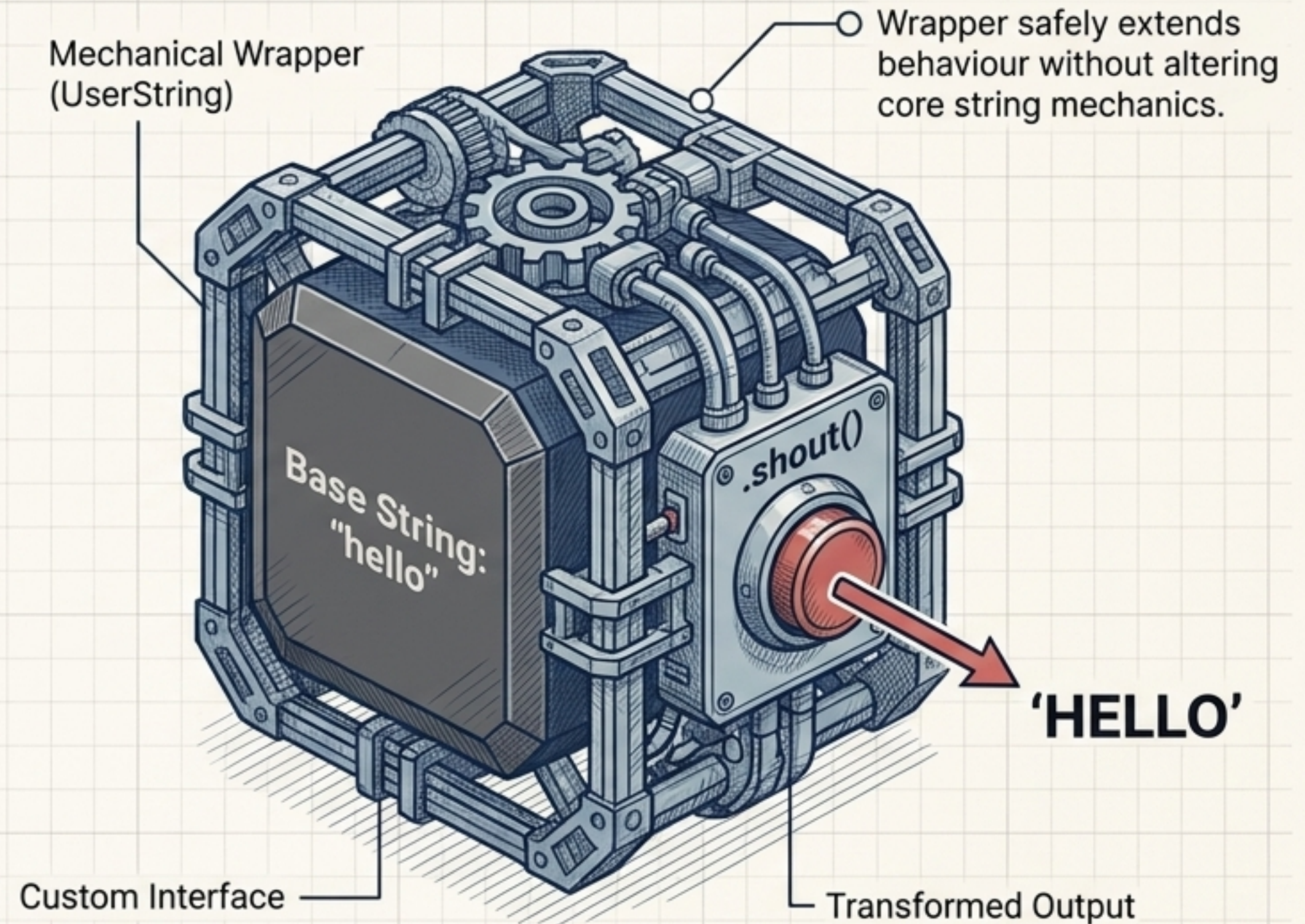
UserString | Transforming Text Objects

Attaching custom, domain-specific operations safely to base text types.

```
from collections import UserString

class ShoutString(UserString):
    def shout(self):
        return self.data.upper()

text = ShoutString('hello')
text.shout() # Returns 'HELLO'
```



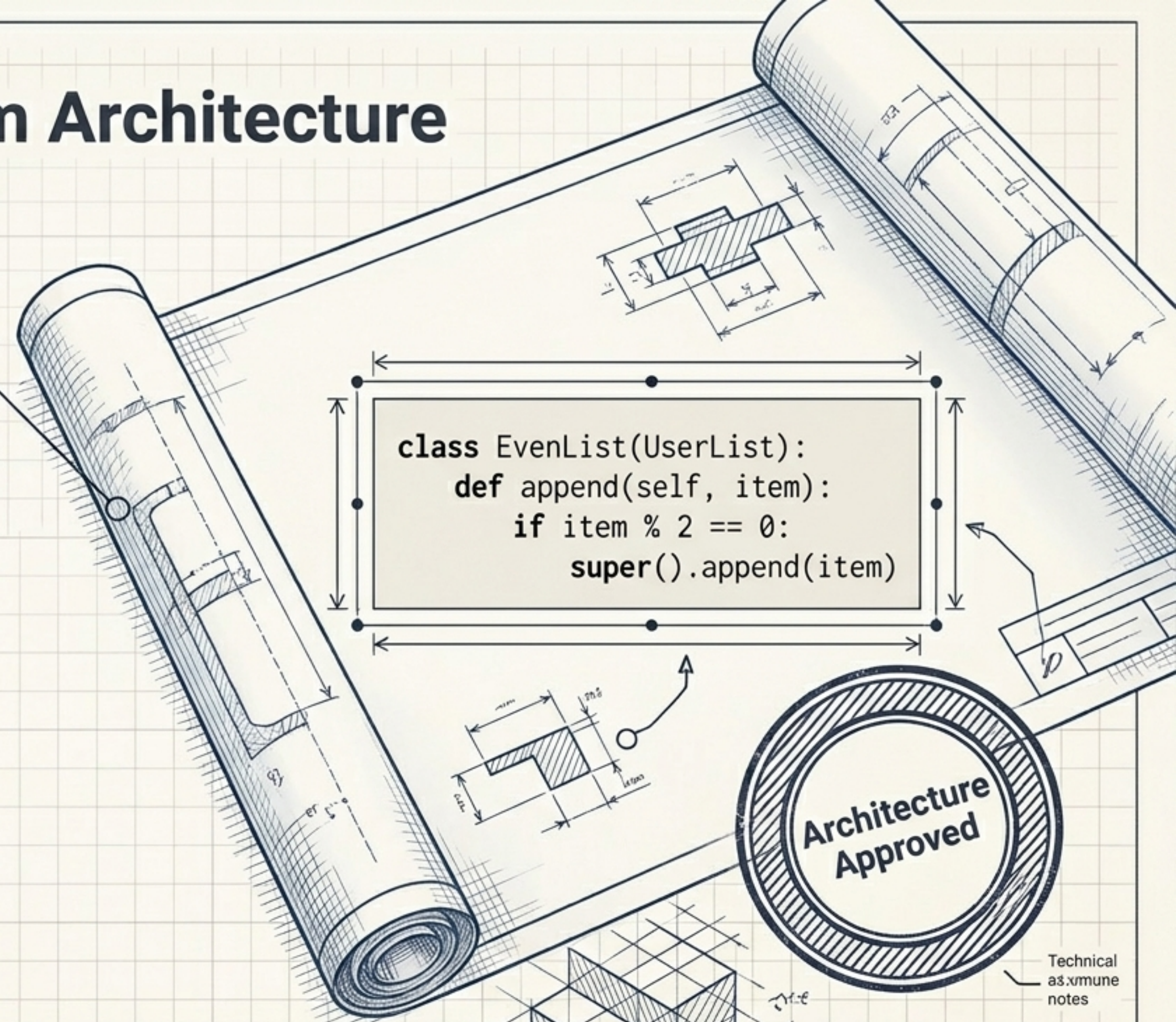
The Value of Custom Architecture

Why engineer custom collections?

Encapsulate Logic: ○
Keep validation rules inherently tied to the data structure.

Ensure Safety:
Extend built-in behaviour without risking core system instability.

Maintainability (DRY):
Promote code reuse and eliminate duplicate validation checks.



```
class EvenList(UserList):  
    def append(self, item):  
        if item % 2 == 0:  
            super().append(item)
```

Architecture
Approved